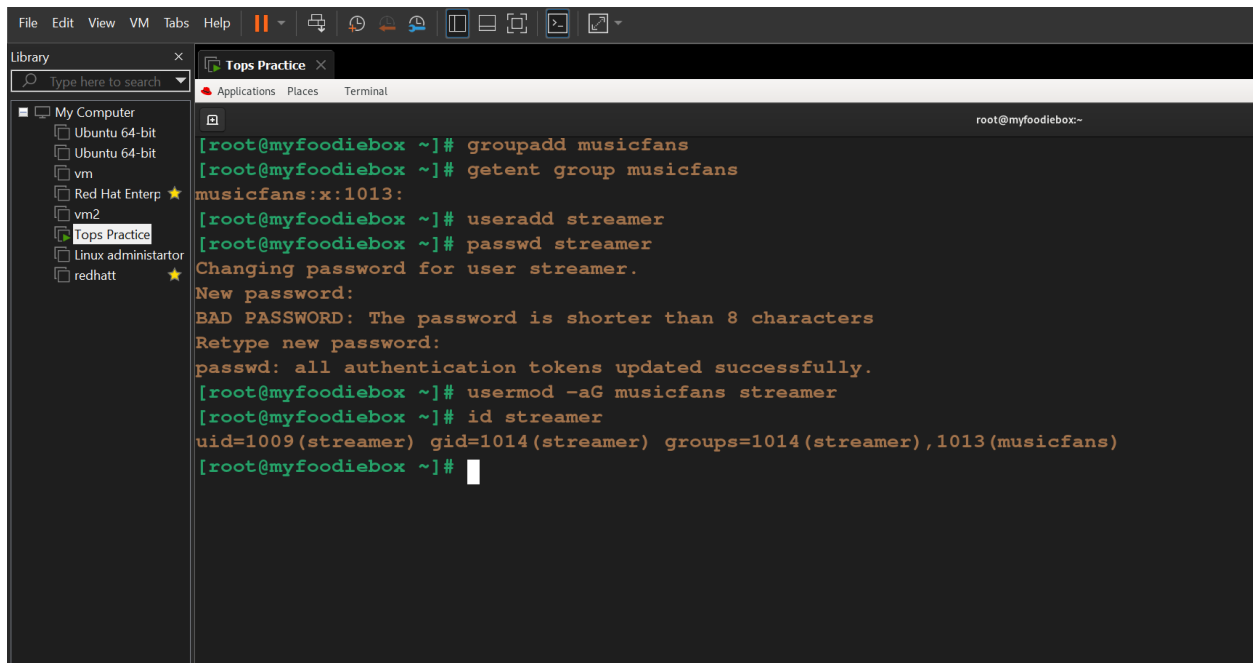


Tasks

1.

Create a new user called streamer and a group called musicfans on your Linux system, then add streamer to the musicfans group. Verify the group membership using the id command.

>



The screenshot shows a terminal window titled 'Tops Practice' with a root prompt. The user is performing the following commands:

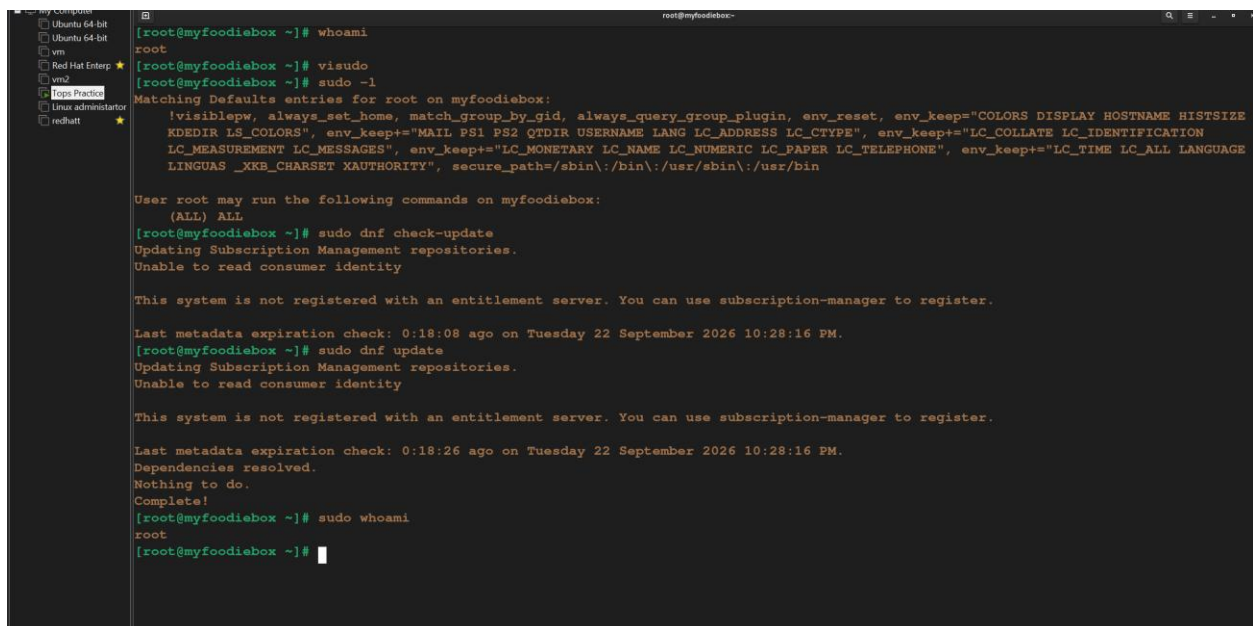
```
[root@myfoodiebox ~]# groupadd musicfans
[root@myfoodiebox ~]# getent group musicfans
musicfans:x:1013:
[root@myfoodiebox ~]# useradd streamer
[root@myfoodiebox ~]# passwd streamer
Changing password for user streamer.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@myfoodiebox ~]# usermod -aG musicfans streamer
[root@myfoodiebox ~]# id streamer
uid=1009(streamer) gid=1014(streamer) groups=1014(streamer),1013(musicfans)
[root@myfoodiebox ~]#
```

The terminal window has a sidebar on the left with a 'Library' section containing a search bar and a list of virtual machines: 'My Computer', 'Ubuntu 64-bit', 'vmn', 'Red Hat Enterp', 'vm2', 'Tops Practice' (selected), 'Linux administartor', and 'redhatt'.

2.

Give your current user sudo privileges by editing the /etc/sudoers file using visudo. Test by running a command that requires sudo, such as updating the package list.

>



The screenshot shows a terminal window with a sidebar on the left containing a list of virtual machines: 'Ubuntu 64-bit', 'Ubuntu 64-bit', 'vm', 'Red Hat Enterp', 'vm2', 'Tops Probes', 'Linux administrator', and 'redhatt'. The main terminal area shows a root shell prompt '[root@myfoodiebox ~]#'. The user has entered 'whoami', which returns 'root'. Then, they entered 'visudo', which shows the sudo configuration for the 'root' user, including a list of allowed commands and environment variables. Finally, they entered 'sudo -l', which shows the matching Defaults entries for the root user. The terminal output is as follows:

```
[root@myfoodiebox ~]# whoami
root
[root@myfoodiebox ~]# visudo
[root@myfoodiebox ~]# sudo -l
Matching Defaults entries for root on myfoodiebox:
!visiblepw, always_set_home, match_group_by_gid, always_query_group_plugin, env_reset, env_keep="COLORS DISPLAY HOSTNAME HISTSIZE
KDEDIR LS_COLORS", env_keep+="MAIL PS1 PS2 QTDIR USERNAME LANG LC_ADDRESS LC_CTYPE", env_keep+="LC_COLLATE LC_IDENTIFICATION
LC_MEASUREMENT LC_MESSAGES", env_keep+="LC_MONETARY LC_NAME LC_NUMERIC LC_PAPER LC_TELEPHONE", env_keep+="LC_TIME LC_ALL LANGUAGE
LINGUAS _XKB_CHARSET XAUTHORITY", secure_path="/sbin:/bin:/usr/sbin:/usr/bin

User root may run the following commands on myfoodiebox:
(ALL) ALL
[root@myfoodiebox ~]# sudo dnf check-update
Updating Subscription Management repositories.
Unable to read consumer identity

This system is not registered with an entitlement server. You can use subscription-manager to register.

Last metadata expiration check: 0:18:08 ago on Tuesday 22 September 2026 10:28:16 PM.
[root@myfoodiebox ~]# sudo dnf update
Updating Subscription Management repositories.
Unable to read consumer identity

This system is not registered with an entitlement server. You can use subscription-manager to register.

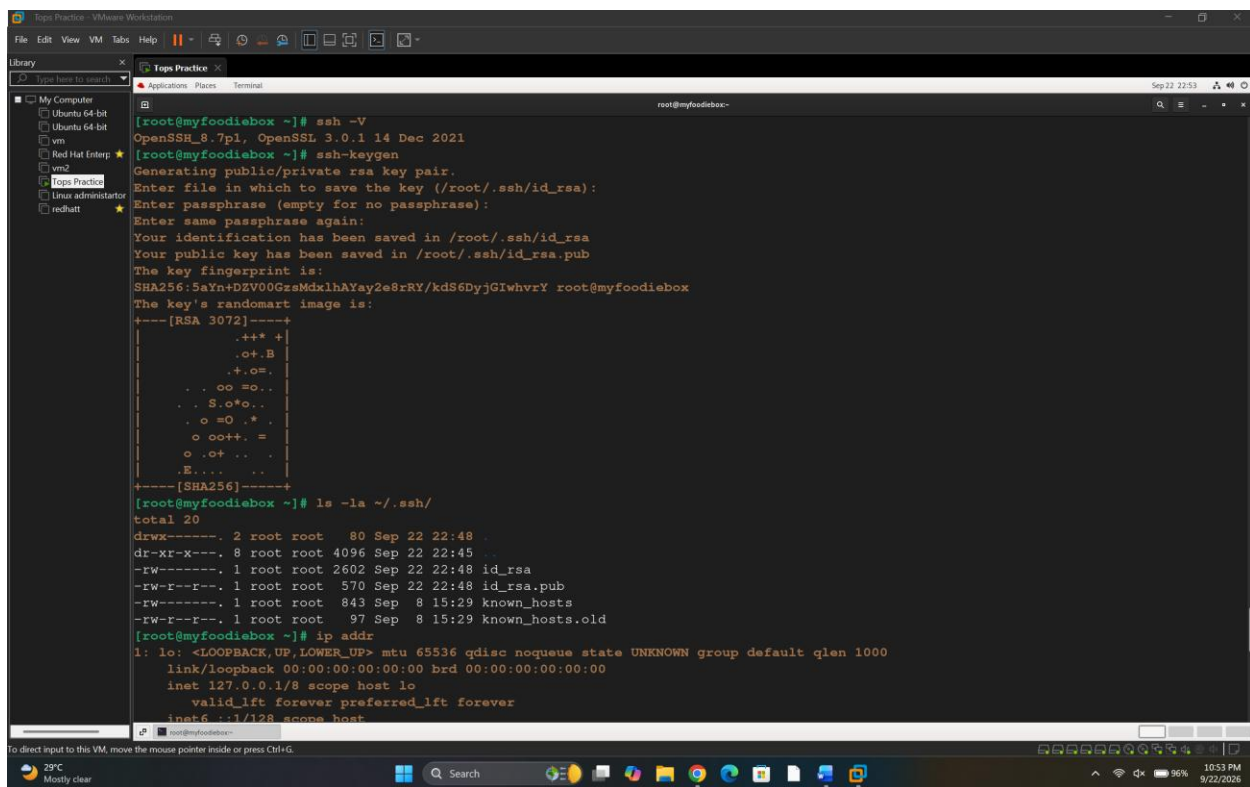
Last metadata expiration check: 0:18:26 ago on Tuesday 22 September 2026 10:28:16 PM.
Dependencies resolved.
Nothing to do.
Complete!
[root@myfoodiebox ~]# sudo whoami
root
[root@myfoodiebox ~]#
```

3.

Set up SSH key-based authentication: generate an SSH key pair using `ssh-keygen`, then copy your public key to a remote Linux server (or a local VM) using `ssh-copy-id`. Disable password authentication for SSH in `/etc/ssh/sshd_config` and restart the SSH service.

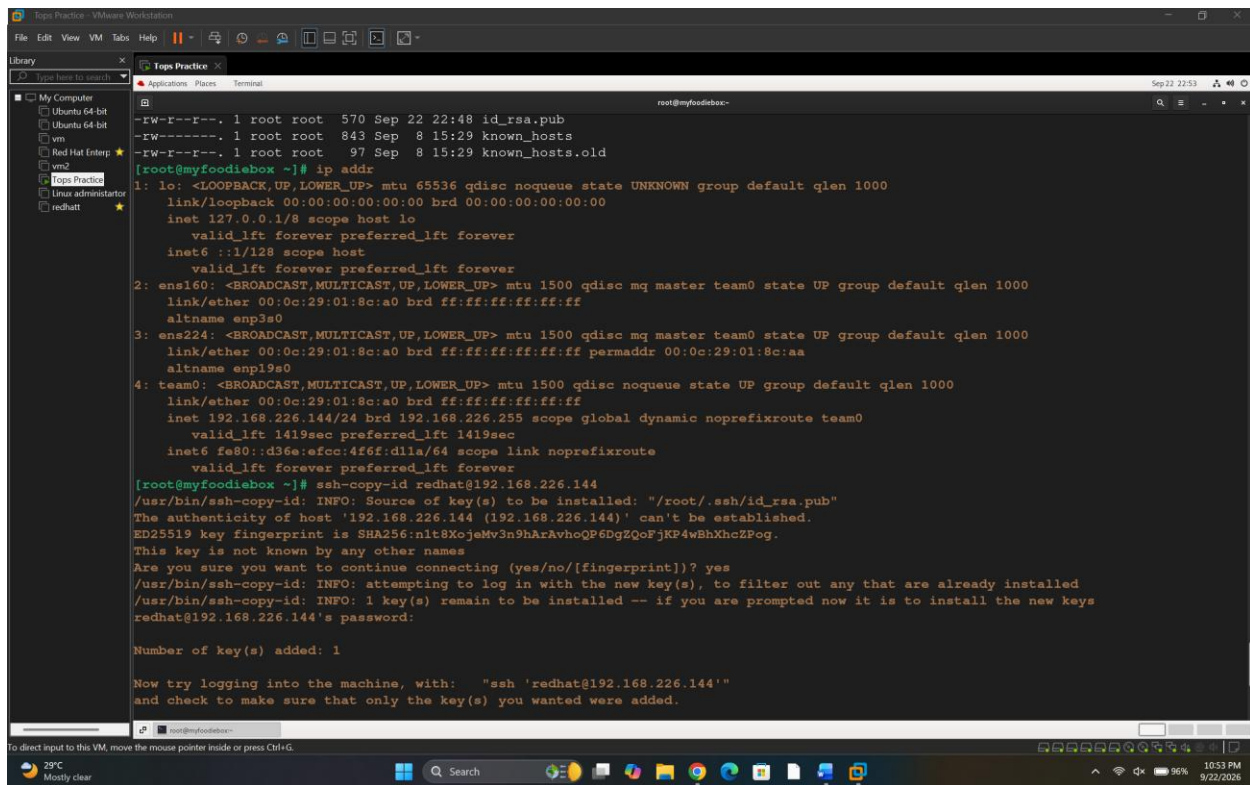
Hint: Look for the `PasswordAuthentication` setting in the SSH config file.

>



The screenshot shows a VMware Workstation window with a terminal running on a VM named 'myfoodiebox'. The terminal output shows the user running 'ssh -V' and 'ssh-keygen' to generate an RSA key pair. The key is saved in '/root/.ssh/id_rsa' and its fingerprint is displayed. The user then runs 'ls -la ~/.ssh/' showing the contents of the .ssh directory. Finally, the user runs 'ip addr' showing the network configuration of the VM.

```
[root@myfoodiebox ~]# ssh -V
OpenSSH_8.7p1, OpenSSL 3.0.1 14 Dec 2021
[root@myfoodiebox ~]# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:5aYn+DZV00GrsMdxlhAYay2e8rRY/kdS6DyJGIwhvY root@myfoodiebox
The key's randomart image is:
+--[RSA 3072]-----+
|.+++.|
|.o+.B|
|.+.o=|
|.oo=oo|
|.S.o*o|
|.o=O.*|
|.oo+.=|
|.o+..|
|.E....|
+--[SHA256]-----+
[root@myfoodiebox ~]# ls -la ~/.ssh/
total 20
drwx----- 2 root root 80 Sep 22 22:48
dr-xr-x--- 8 root root 4096 Sep 22 22:45
-rw----- 1 root root 2602 Sep 22 22:48 id_rsa
-rw-r--r-- 1 root root 570 Sep 22 22:48 id_rsa.pub
-rw----- 1 root root 843 Sep 8 15:29 known_hosts
-rw-r--r-- 1 root root 97 Sep 8 15:29 known_hosts.old
[root@myfoodiebox ~]# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
```

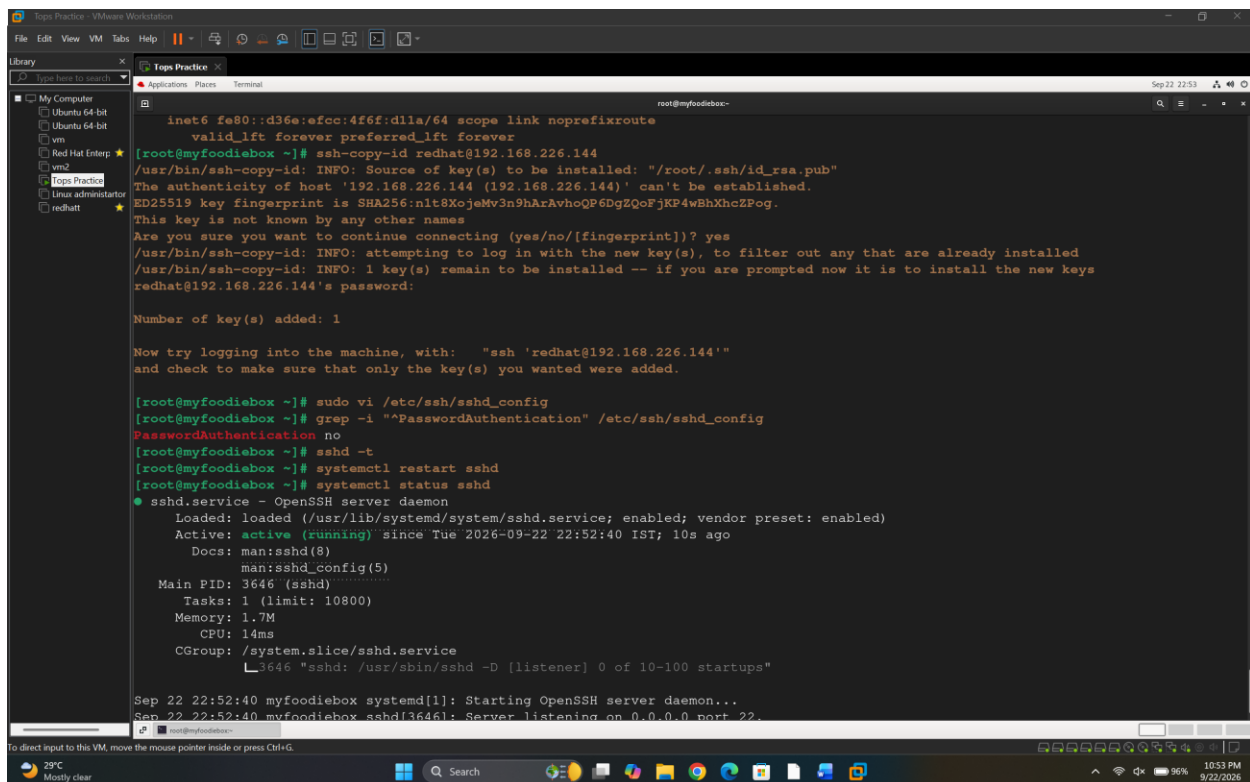


The screenshot shows a VMware Workstation window with a terminal running on a VM named 'myfoodiebox'. The terminal output shows the user running 'ssh-copy-id redhat@192.168.226.144' to copy the public key to the remote host. The output shows the key being added and the user being prompted to confirm the connection. The user then runs 'ssh 'redhat@192.168.226.144'' to log in to the remote host.

```
-rw-r--r-- 1 root root 570 Sep 22 22:48 id_rsa.pub
-rw----- 1 root root 843 Sep 8 15:29 known_hosts
-rw-r--r-- 1 root root 97 Sep 8 15:29 known_hosts.old
[root@myfoodiebox ~]# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq master team0 state UP group default qlen 1000
    link/ether 00:0c:29:01:8c:a0 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
3: ens224: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq master team0 state UP group default qlen 1000
    link/ether 00:0c:29:01:8c:a0 brd ff:ff:ff:ff:ff:ff permaddr 00:0c:29:01:8c:aa
    altname enp19s0
4: team0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 00:0c:29:01:8c:a0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.226.144/24 brd 192.168.226.255 scope global dynamic noprefixroute team0
        valid_lft 1419sec preferred_lft 1419sec
    inet6 fe80::d36e:efcc:4f6f:d11a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
[root@myfoodiebox ~]# ssh-copy-id redhat@192.168.226.144
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_rsa.pub"
The authenticity of host '192.168.226.144 (192.168.226.144)' can't be established.
ED25519 key fingerprint is SHA256:nlt8XojeHv3n9hArAvhcQP6DgZQoFjKp4wBhXhcZPog.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
redhat@192.168.226.144's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'redhat@192.168.226.144'"
and check to make sure that only the key(s) you wanted were added.
```



```
root@myfoodiebox:~# ssh-copy-id redhat@192.168.226.144
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_rsa.pub"
The authenticity of host '192.168.226.144 (192.168.226.144)' can't be established.
ED25519 Key fingerprint is SHA256:nlt8XojeHv3n9hArAvhoQP6DgZQoFjKp4wBhXhcZPog.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
redhat@192.168.226.144's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'redhat@192.168.226.144'"
and check to make sure that only the key(s) you wanted were added.

[root@myfoodiebox ~]# sudo vi /etc/ssh/sshd_config
[root@myfoodiebox ~]# grep -i "^PasswordAuthentication" /etc/ssh/sshd_config
PasswordAuthentication no
[root@myfoodiebox ~]# sshd -t
[root@myfoodiebox ~]# systemctl restart sshd
[root@myfoodiebox ~]# systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2026-09-22 22:52:40 IST; 10s ago
     Docs: man:sshd(8)
           man:sshd_config(5)
   Main PID: 3646 (sshd)
      Tasks: 1 (limit: 10800)
     Memory: 1.7M
        CPU: 14ms
    CGroup: /system.slice/ssh.service
            └─3646 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Sep 22 22:52:40 myfoodiebox systemd[1]: Starting OpenSSH server daemon...
Sep 22 22:52:40 myfoodiebox sshd[3646]: Server listening on 0.0.0.0 port 22.
```

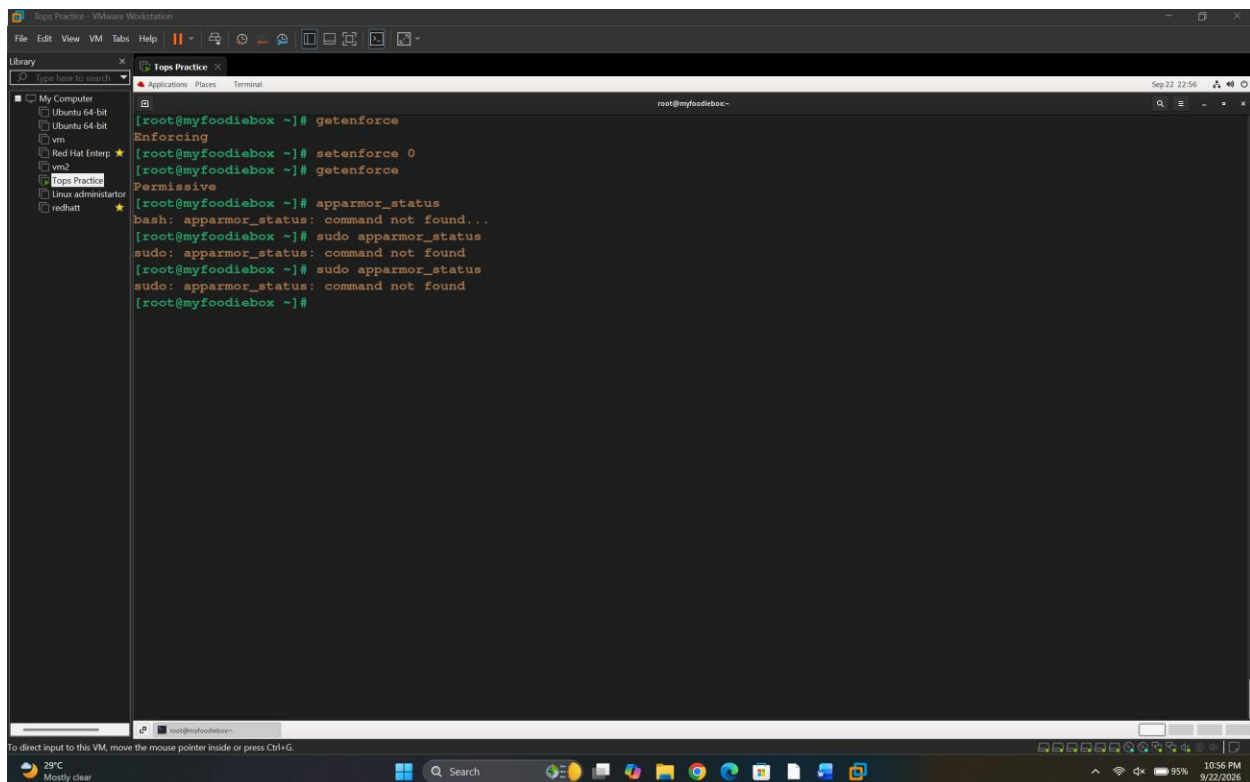
```
[root@myfoodiebox ~]# sudo vi /etc/ssh/sshd_config
[root@myfoodiebox ~]# grep -i "^PasswordAuthentication" /etc/ssh/sshd_config
PasswordAuthentication no
[root@myfoodiebox ~]# sshd -t
[root@myfoodiebox ~]# systemctl restart sshd
[root@myfoodiebox ~]# systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2026-09-22 22:52:40 IST; 10s ago
     Docs: man:sshd(8)
           man:sshd_config(5)
   Main PID: 3646 (sshd)
      Tasks: 1 (limit: 10800)
     Memory: 1.7M
        CPU: 14ms
    CGroup: /system.slice/ssh.service
            └─3646 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Sep 22 22:52:40 myfoodiebox systemd[1]: Starting OpenSSH server daemon...
Sep 22 22:52:40 myfoodiebox sshd[3646]: Server listening on 0.0.0.0 port 22.
Sep 22 22:52:40 myfoodiebox sshd[3646]: Server listening on :: port 22.
Sep 22 22:52:40 myfoodiebox systemd[1]: Started OpenSSH server daemon.
[root@myfoodiebox ~]#
[root@myfoodiebox ~]#
```

4.

Check if SELinux or AppArmor is enabled on your system. If SELinux is active, use the `getenforce` command to display its current mode, and switch it to permissive mode. If AppArmor is active, list all enforced profiles using `sudo apparmor_status`.

>



5.

Use ChatGPT to explain the difference between SELinux and AppArmor in one paragraph, then write a summary in your own words and mention one real-world app (like Zomato or Paytm) that would benefit from these security modules.

>

SELinux and AppArmor are both Linux security systems that use mandatory access control to restrict what applications and processes can access. SELinux mainly uses security labels and policies to control access to files, processes, ports, and other system resources, while AppArmor uses application-based profiles that define what a particular program is allowed to access. SELinux is commonly used on Red Hat Enterprise Linux, while AppArmor is commonly associated with Ubuntu and other distributions. Both can reduce the impact of a compromised application by limiting what that application can do.

Summary in My Own Words

SELinux and AppArmor both provide additional security to Linux systems. They control what applications and processes are allowed to access. SELinux uses security labels and policies, while AppArmor uses profiles for applications. RHEL commonly uses SELinux, whereas Ubuntu commonly uses AppArmor.

Real-World Application

A large application platform such as Zomato can use Linux security controls such as SELinux or AppArmor on its servers. These security modules can restrict application processes so that a compromised application has limited access to files, services, and other system resources.